

Lecture 1: Your First Day as a Coder

Setup and the building blocks: Python, VS Code, venv, pip, Variables, Data Types, Type Casting, Operators

Module	Duration	For
M1, Python Foundations (Day 1 of 9)	2 hours (120 min)	Absolute beginners, zero coding needed

How to use this workbook

This workbook is for doing, not just reading. Keep your laptop open. When you see **TYPE THIS**, type the code and run it yourself. When you see **PREDICT**, first write down what you think the answer will be, then run the code and check. It is fine to be wrong, that is how you learn. We follow the same simple order all day: **first see it work, then learn why, then learn the rules**.

What you will be able to do by the end of today

- Install Python and VS Code, and run a program you wrote yourself.
- Say what a **variable** is in one line, and use one with confidence.
- Know the four common data types, **int**, **float**, **str**, **bool**, and pick the right one.
- Change (**cast**) data from one type to another, and know why code crashes without it.
- Do maths and simple decisions with **operators**, and build a small bill calculator.

THE MAP, WHERE TODAY SITS

A **full-stack** app is like a restaurant. The **front end** is the dining hall the customer sees (menu, tables, lights). The **back end** is the kitchen they do not see (cooking, billing). The **database** is the store-room where everything is kept.

Over 9 weeks you will build the full restaurant. But first you learn the basics. Variables, data types and operators are the small building blocks. **Every line of code is made of them**, front end, back end, and AI. So we start here.

Today's 2-hour plan

Time	What we do	Why now
0:00 to 0:10	Welcome + the big picture	Know where we are headed
0:10 to 0:30	Install Python + VS Code, run 'Hello'	Everyone tastes success early
0:30 to 0:42	venv & pip, project ka alag dabba	Clean habits from Day 1
0:42 to 1:02	Variables	The first real idea
1:02 to 1:27	Data types (int, float, str, bool)	What can live in a variable
1:27 to 1:45	Type casting	Fix the most common beginner crash
1:45 to 2:00	Operators + mini-project + quiz	Put it all together

1 Getting Set Up: From an Empty Laptop to Hello

0:10 to 0:30 | 20 min

First, the idea

Think about a restaurant menu. We can only order the dishes already printed on it. We cannot ask for a dish that is not on the menu. Apps like WhatsApp, games, or YouTube are like that menu: we can only do the things they already offer.

Coding is different. It is like cooking in our own kitchen, where we decide what to make and how. With code, we give the computer our own instructions, and it follows them. To cook, we need two things: a **gas stove** to cook on, and a clean **kitchen platform** to work on. Here, **Python is the stove** and **VS Code is the platform**.

Step 1, Install Python (the stove)

Python is the language we will use to talk to the computer. Go to **python.org/downloads**, download the latest version, and run the installer.

Don't miss this one checkbox

On the first installer screen (Windows), tick the box 'Add Python to PATH' before you click Install.

Why does this matter? Think of our phone contacts. We save our mother's number once as 'Mummy'. After that, we just tap 'Mummy' to call; we do not type the full number again.

'Add to PATH' does the same job for Python. After you tick it, you can type just `python` from anywhere and the computer knows what you mean. If you skip it, the computer cannot find Python. (On Mac this is usually done for you.)

Check it worked. Open your terminal (Command Prompt on Windows, Terminal on Mac) and type:

```
python --version  
# You should see something like: Python 3.12.4
```

Step 2, Install VS Code (the platform)

VS Code is where you write your code. Download it from **code.visualstudio.com** and install it. Then open VS Code, click the squares icon on the left, search for '**Python**' (made by Microsoft), and click Install. This adds a Run button and helpful hints.

Step 3, Run your first program

Make a folder named day1. Open it in VS Code (**File > Open Folder**). Make a new file called `hello.py` and type this:

```
print("Namaste, World!")  
print("My name is Bipul and today I wrote code.")
```

The .py at the end tells the computer it is a Python file. print() shows text on the screen.

Now run it: click the **Run** button at the top-right (or right-click and choose *Run Python File*). Your two lines appear in the panel at the bottom. **That panel is the computer replying to you.**

What print() does

Code without `print()` is like solving a sum in your head, the answer is there, but you cannot see it. `print()` tells the computer to **show the answer on screen**. So whenever you want to see what your code did, put it inside `print()`.

Common first-day errors (everyone gets these)

- **'python' is not recognised**, you missed the 'Add to PATH' box. Install Python again and tick it.
- **SyntaxError**, you forgot a bracket or a quote. Python is strict, like a TTE checking train tickets: every (needs a), and every " needs a closing ".
- **Nothing happens**, you ran the wrong file. Check the file name at the top of the tab.

2 venv and pip: Give Each Project Its Own Dabba

0:30 to 0:42 | 12 min

First, the problem

Imagine one shared dabba (tiffin box) in the hostel mess where **everyone** puts their food. Your poha, someone's biryani, someone's kheer, all mixed together. No meal tastes right.

Code has the same problem. One project needs version 1 of a tool. Another project needs version 2. If all projects share one big dabba, the versions clash and break. The fix is simple: **give each project its own clean dabba**. This clean dabba is called a **virtual environment**, or **venv** for short.

Step 1: make the dabba

In VS Code, open the terminal (the **Terminal** menu, then **New Terminal**). Make sure you are inside your day1 folder. Then type these two lines:

```
# create the dabba (a hidden folder named .venv)
python -m venv .venv

# turn the dabba on (this is called activating it)
# on Windows:
.venv\Scripts\activate
# on Mac or Linux:
source .venv/bin/activate
```

After this you will see (.venv) at the start of the terminal line. That means the dabba is on.

Step 2: tell VS Code to use this dabba

Making the dabba is not enough. When you press the Run button, VS Code might still use some other Python. So you tell it once which dabba to use:

- Press `Ctrl+Shift+P` (on Mac, `Cmd+Shift+P`) to open the command box at the top.
- Type **Python: Select Interpreter** and click it.
- From the list, pick the one that has `.venv` in its name (it usually says 'Recommended').

Done. Now both the Run button and the terminal use your dabba. You can check it any time: look at the bottom bar of VS Code, it shows the Python in use, and it should mention `.venv`.

Step 3: pip brings ready-made tools

A **library** is ready-made code that someone else has already written, so we do not have to write it ourselves. **pip** is the tool that downloads a library and puts it into our active dabba, like Zepto or Blinkit brings groceries to our door.

Let us try a small, fun library called **cowsay**. First, download it:

```
pip install cowsay      # download the cowsay library
```

Because the dabba is on, pip puts cowsay inside .venv, so our other projects stay clean.

Now make a new file and use it. We did not write this cow; cowsay did. We just use it:

```
import cowsay            # bring in the tool we just downloaded
cowsay.cow("Python is fun!")
```

Run it, and a cartoon cow appears on screen saying 'Python is fun!'. That is the power of pip: borrow code others wrote.

A few more you can try the same way: `pip install pyjokes` (tells programming jokes), or `pip list` to see every library inside the dabba right now.

You may hear about requirements.txt. Not needed today.

`requirements.txt` is just a text file with a list of the libraries a project uses. The command `pip freeze > requirements.txt` looks at whatever is installed in the dabba that is on **right now**, and writes those names into that file. It does not magically know the project; it only reads the active dabba.

So if we ran it today, after only a `print()` and installing just cowsay, the file would have almost nothing in it. That is fine. This file becomes useful later, when we build a real project that uses many libraries. We will use it properly then.

3 Variables: Names That Remember Things

0:42 to 1:02 | 20 min

First, the idea

We already use this idea every day. To call our mother, we do not dial **+91 98765 43210**. We just tap '**Mummy**' in our contacts. The name 'Mummy' stands for the number. If her number changes, we update it once, and 'Mummy' still works.

A variable is the same idea in code. It is a short **name** that stands for a **value**. We use the name, and the computer brings back the value.

Let us make some. Type this and run it:

```
name = "Bipul"  
age = 19  
city = "Hyderabad"  
  
print(name)  
print("Age:", age)  
print(name, "lives in", city)
```

The = sign means "put into". So name = "Bipul" means: put the text Bipul into the name box.

Picture a dabba with a label

Think of a steel dabba with a label stuck on it. The **label** is the variable name (age). What is **inside** is the value (19). You can throw out what is inside and put something new, and the label stays the same:

First age = 19, and later age = 20. Same label, new value. Changing the value like this is normal and allowed.

The most common beginner mix-up: = and ==

= (one equals) An order: 'put this value in.' <code>marks = 90</code> <i>It does something.</i>	== (two equals) A question: 'are these two equal?' <code>marks == 90</code> <i>It answers True or False.</i>
---	--

Rules for naming a variable

- Use clear names like `student_name`, not short ones like `x`. It helps you read your code later.
- Use small letters with underscores, like `total_marks`. This is the normal Python style.
- Capital and small letters are different. `Name` and `name` are two different variables.
- No spaces, and it cannot start with a number. `2nd_rank` is wrong; `rank_2` is right.

Predict, then run

First write down what you think will print. Then type the code, run it, and check.

Code	Your guess
<pre>x = 5 x = 8 print(x)</pre>	
<pre>a = "10" print(a) print(Age)</pre>	

Hint for the second one: we never made a variable called Age with a capital A. What do you think the computer will say?

4 Data Types: What Kind of Value Is Inside?

1:02 to 1:27 | 25 min

First, the idea

In a kirana shop, different things are kept in different ways. **Rice** goes in a sack and you weigh it. **Eggs** go in a tray and you count them. **The shop name** is painted on a board, so it is text. **'Is the shutter open?'** has only two answers: yes or no.

Code works the same way. The **kind** of a value decides what you can do with it. There are four kinds you will use every day. Here they are:

Type	It holds	Everyday example	In code
int	Whole numbers you count	Rotis on a plate, students present	<code>count = 4</code>
float	Numbers with a decimal point	Petrol Rs 104.57, CGPA 8.7	<code>price = 104.57</code>
str	Text, always in quotes	A name, a WhatsApp message	<code>name = "Asha"</code>
bool	Only True or False	Is the fan on? Is the gate open?	<code>is_on = True</code>

Run this and read the output. `type()` asks Python: what kind of value is this?

```
rotis = 4
price = 104.57
name = "Asha"
shutter_open = True

print(type(rotis))      # <class 'int'>
print(type(price))     # <class 'float'>
print(type(name))      # <class 'str'>
print(type(shutter_open)) # <class 'bool'>
```

A common trap: numbers that are really text

A **mobile number** and a **PIN code** look like numbers, but you should store them as **str**, not **int**. Here is why. You never do maths on them; you never add two phone numbers. Also, **int** would drop a starting zero, so a PIN like 06210 would wrongly become 6210. Easy rule:

Will you do maths on it? Use a number (int or float). Is it just a label or an ID? Use text (str).

Why does the type matter? Watch the + sign behave differently.

The + sign does different jobs for different types. It is like the word 'bat': it can mean a cricket bat or an animal. Same word, different meaning. Same here:

```
print(5 + 3)      # 8      numbers get added
print("5" + "3")  # 53     text gets joined together
print("Chai" + " " + "Garam") # Chai Garam
```

This is not a bug. It is the type doing its job. Numbers get added. Text gets joined. Joining text like this is called **concatenation**. Most beginner errors come from mixing up the type.

Predict: int, float, or str? And what prints?

Code	Your guess
<pre>pin = "560001" print(type(pin))</pre>	
<pre>print(10 + 5) print("10" + "5")</pre>	
<pre>cgpa = 8.7 print(type(cgpa))</pre>	

5 Type Casting: Changing One Type Into Another

1:27 to 1:45 | 18 min

First, a crash you will see

Type this and run it. It **will** crash, and that is exactly the point:

```
age = input("Enter your age: ")    # you type 19
print("Next year you will be", age + 1)
```

The error says: can only concatenate str (not "int") to str. Python is trying to tell you something useful.

Here is the key point every beginner must learn: `input()` **always gives you text (str)**, even when you type digits. So age is the text "19", not the number 19. You cannot add 1 to text. First you must change it into a number. Changing a value from one type to another is called **type casting**.

Think of the airport money counter

You go abroad with Rs 5000 in your pocket. A vending machine there will not take rupees; it wants dollars. The money is the same, but the machine needs it in a different form. So you go to the exchange counter and change rupees into dollars.

Type casting is that exchange counter. `int("19")` changes the text "19" into the number 19. Same value, new form, now ready for maths.

The fix: change the input into an int before doing maths.

```
age = input("Enter your age: ")    # age is the text "19"
age = int(age)                    # change "19" into the number 19
print("Next year you will be", age + 1)    # now it works, prints 20
```

You can also write it in one line: `age = int(input("Enter your age: "))`

The four change tools

Tool	Changes into	Example
<code>int(x)</code>	whole number	<code>int("19")</code> gives 19
<code>float(x)</code>	decimal number	<code>float("8.7")</code> gives 8.7
<code>str(x)</code>	text	<code>str(20)</code> gives "20"
<code>bool(x)</code>	True or False	<code>bool(0)</code> gives False

Casting works the other way too. To put a number *inside* a sentence joined with `+`, change it into text first, or the types will clash:

```
score = 90
# print("Score: " + score)    # this crashes: text + number
print("Score: " + str(score)) # works, prints Score: 90
print("Score:", score)        # also works, using a comma
```

Careful: not everything can be changed

`int("hello")` crashes, because there is no number inside the word 'hello', so the counter refuses. `int("19.5")` also crashes, because 19.5 is not a whole number; use `float("19.5")` instead. The counter only changes a value when it really can.

6 Operators: Doing Maths and Making Decisions

1:45 to 2:00 | 15 min

An **operator** is a symbol that does a small job, like the buttons on a shop calculator. There are three groups you will use all the time.

a) Arithmetic: the shop calculator

```
print(10 + 3)    # 13    add
print(10 - 3)    # 7     subtract
print(10 * 3)    # 30    multiply
print(10 / 3)    # 3.333... divide (always a decimal)
print(10 // 3)   # 3     floor divide (keep only the whole part)
print(10 % 3)    # 1     modulo (the remainder left over)
print(10 ** 3)   # 1000  power (10 multiplied by itself 3 times)
```

// and % at the chai tapri

You have **Rs 35**. One cup of chai costs **Rs 10**. How many full cups can you buy, and how much change is left?

`35 // 10` gives **3 cups** (floor divide keeps only whole cups; the shop will not sell half a chai). `35 % 10` gives **Rs 5 left** (modulo is the change left over).

Useful trick: % tells you if a number is even or odd. `n % 2 == 0` means it divides by 2 with no remainder, so the number is **even**.

b) Comparison: questions that answer True or False

These compare two values and give back a **bool** (True or False). Good for checking if someone qualifies:

```

age = 19
print(age >= 18)    # True    old enough to vote
print(age == 21)    # False   not exactly 21
print(age != 18)    # True    not equal to 18
marks = 30
print(marks >= 33) # False   did not pass

```

c) Logical: joining conditions with and, or, not

Operator	True when	Everyday rule
and	BOTH sides are True	To board a flight you need a ticket and an ID
or	AT LEAST ONE side is True	You can pay by UPI or cash; either one works
not	Flips True to False, and False to True	If it is not raining, step out

```

has_id    = True
before_10pm = False
# Hostel rule: you may enter only with an ID and before 10pm
print(has_id and before_10pm)    # False, because it is late
print(has_id or before_10pm)     # True, or needs just one
print(not before_10pm)           # True, it is not before 10pm

```

7 Put It Together: The Chai Tapri Bill Calculator

Mini-project

This small program uses **everything from today**: input, variables, casting, data types and operators. Type it, run it, then change the numbers and run it again.

```
# Chai Tapri Bill
name = input("Customer name: ")           # text
cups = int(input("How many cups of chai? ")) # change to a number
rate = 10                                 # price per cup
total = cups * rate                       # arithmetic
is_big_order = cups >= 5                  # comparison, gives True or False
print("Bill for " + name)                 # joining text
print("Total: Rs", total)                  # comma, no change needed
print("Bulk customer?", is_big_order)      # prints True or False
```

Try 6 cups, then try 2 cups, and watch the True or False answer change.

Your turn (try these after class)

1. Add a samosas input at Rs 15 each, and show the combined total.
2. Give a Rs 5 discount only when the order is big (5 cups or more).
Hint: work out the total, then subtract.
3. Print the full bill as one clean sentence, using `str()` to join the numbers.

Quick Quiz, 6 questions (answers on the last page)

1. What does `type(8.7)` give: int, float, str or bool?
2. What prints for `print("7" + "2")`? And for `print(7 + 2)`?
3. Why should a mobile number be stored as a str, not an int? Give one reason.
4. `input()` always gives which data type?
5. You have Rs 47 and chai costs Rs 10. What do `47 // 10` and `47 % 10` give?
6. What is the difference between `=` and `==`?

Coming up in Day 2: Control Flow

Today your programs ran from top to bottom, every line, in order, like a train on a single track. But real life needs **decisions** and **repeating**. For example: if the order is big, give a discount; or keep asking while the customer adds more items.

In Day 2 we give your code the power to **choose** and to **repeat**, using `if`, `elif`, `else`, and loops. The True and False values you learned today are what make those decisions work. See you on Day 2.

Recap Answer Key and One-Page Summary

Quiz answers

- 1. **float**, because it has a decimal point.
- 2. "7"+"2" gives **72** (text is joined). 7+2 gives **9** (numbers are added).
- 3. You never do maths on it, and int would drop a starting zero. It is an ID, so store it as str.
- 4. **str** (text), always. Change it with int() or float() before doing maths.
- 5. 47 // 10 gives **4** cups; 47 % 10 gives **Rs 7** left over.
- 6. = puts a value into a variable. == checks if two values are equal and gives True or False.

Today on one page

Setup	Python is the stove. VS Code is the platform. Tick Add to PATH . print() shows the answer on screen.
venv / pip	Each project gets its own clean dabba (venv). Tell VS Code to use it with Select Interpreter. pip brings tools into the dabba.
Variable	A name that stands for a value, like 'Mummy' in our contacts. = means 'put into'.
Data types	int (count), float (decimals), str (text in quotes), bool (True or False). IDs and labels go as str.

Casting	The money-exchange counter: <code>int()</code> <code>float()</code> <code>str()</code> <code>bool()</code> . <code>input()</code> always gives str.
Operators	Arithmetic (+ - * / // % **), Comparison (gives True or False), Logical (and or not).